ASSIGNMENT 6

Data Integrity Constraints

Event Ticketing and Venue Seating System

Name	Aishani Dutta
Registration Number	2511021240020
Course / Section	BCA SEM 3 2025
Date	23 October 2026

Constraints used

Constraint	Use in this database
Primary Key	Each table has its own identifier: user_id, venue_id, seat_id, event_id, booking_id, booking_seat_id or payment_id.
Foreign Key	Connects users, venues, events, bookings, seats and payments. A child cannot refer to a missing parent.
UNIQUE	Protects phone, email, ticket_code, the physical seat location and the event-seat pair.
CHECK	Rejects invalid status values, negative prices, non-positive payments and non-positive venue capacities.
DEFAULT	Uses the current time for new accounts and bookings, Regular for a seat type, and Pending for a booking or payment.
NOT NULL	Requires essential information such as names, identifiers, event dates and amounts.

The Primary Key and Foreign Key definitions are included in Assignment 5. ON DELETE RESTRICT prevents deleting a parent record while child records still refer to it.

Example: preventing double booking

```
CONSTRAINT one_seat_per_event UNIQUE (event_id, seat_id)
```

Seat 1 can be allocated once for event 101 and once for event 104. It cannot appear twice for event 101. This also protects the seat if two requests arrive at the same time.

Example: matching a ticket to its booking

```
FOREIGN KEY (booking_id, event_id)
    REFERENCES Bookings(booking_id, event_id)
```

If booking 201 is for event 101, a ticket row cannot link booking 201 to event 104. Two independent Foreign Keys would not be enough to check this combination.

Checking the venue

A seat must belong to the venue where its event is held. The following triggers compare the two venue identifiers when a ticket row is inserted or updated. The parent rows are locked while this check runs. Foreign Keys still reject missing records.

```
USE EventTicketingDB;
DELIMITER //
CREATE TRIGGER check_seat_insert
BEFORE INSERT ON Booking_Seats
FOR EACH ROW
BEGIN
    DECLARE seat_venue INT;
    DECLARE event_venue INT;
    SELECT venue_id INTO seat_venue FROM Seats
        WHERE seat_id = NEW.seat_id FOR SHARE;
    SELECT venue_id INTO event_venue FROM Events
        WHERE event_id = NEW.event_id FOR SHARE;
    IF seat_venue IS NOT NULL AND event_venue IS NOT NULL
       AND seat_venue <> event_venue THEN
        SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT = 'Seat belongs to another venue';
    END IF;
END//

CREATE TRIGGER check_seat_update
BEFORE UPDATE ON Booking_Seats
FOR EACH ROW
BEGIN
    DECLARE seat_venue INT;
    DECLARE event_venue INT;
    SELECT venue_id INTO seat_venue FROM Seats
        WHERE seat_id = NEW.seat_id FOR SHARE;
    SELECT venue_id INTO event_venue FROM Events
        WHERE event_id = NEW.event_id FOR SHARE;
    IF seat_venue IS NOT NULL AND event_venue IS NOT NULL
       AND seat_venue <> event_venue THEN
        SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT = 'Seat belongs to another venue';
    END IF;
END//
```

The seat and event venue assignments are kept fixed in this version. To move an event to another venue, create the new event and handle its bookings separately.

```
CREATE TRIGGER keep_seats_venue
BEFORE UPDATE ON Seats
FOR EACH ROW
BEGIN
    IF NEW.venue_id <> OLD.venue_id THEN
        SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT = 'Venue cannot be changed';
    END IF;
END//

CREATE TRIGGER keep_events_venue
BEFORE UPDATE ON Events
FOR EACH ROW
BEGIN
    IF NEW.venue_id <> OLD.venue_id THEN
        SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT = 'Venue cannot be changed';
    END IF;
END//

DELIMITER ;
```

Testing the constraints

The following statements use the sample records from Assignment 7. Each test is run inside START TRANSACTION and ROLLBACK. An error is the expected result for an invalid entry. Run the blocks separately so that an error does not prevent the ROLLBACK statement from being executed.

```
USE EventTicketingDB;
START TRANSACTION;
-- Run one test and inspect its output.
ROLLBACK;
```

1. Duplicate Primary Key

```
INSERT INTO Venues VALUES
(1, 'Test Hall', 'Test Road', 'Pune', 6);
```

ERROR 1062: Duplicate entry '1' for key 'venues.PRIMARY'

2. Missing parent record

```
INSERT INTO Payments
(payment_id, booking_id, amount, payment_method)
VALUES (999, 999, 100, 'UPI');
```

ERROR 1452: Cannot add or update a child row: a foreign key constraint fails (`eventticketingdb`.`payments`, CONSTRAINT `payments_ibfk_1` FOREIGN KEY (`booking_id`) REFERENCES `bookings` (`booking_id`) ON DELETE RESTRICT)

3. Duplicate email

```
UPDATE Users SET email = 'ananya@example.com'
WHERE user_id = 2;
```

ERROR 1062: Duplicate entry 'ananya@example.com' for key 'users.email'

4. Duplicate ticket code

```
UPDATE Booking_Seats SET ticket_code = 'TKT-301'
WHERE booking_seat_id = 302;
```

ERROR 1062: Duplicate entry 'TKT-301' for key 'booking_seats.ticket_code'

5. Seat already booked for the event

```
INSERT INTO Booking_Seats VALUES
(999, 201, 1, 101, 500, 'TEST-999');
```

ERROR 1062: Duplicate entry '101-1' for key 'booking_seats.one_seat_per_event'

6. Booking and ticket refer to different events

```
INSERT INTO Booking_Seats VALUES
(999, 201, 3, 104, 500, 'TEST-999');
```

ERROR 1452: Cannot add or update a child row: a foreign key constraint fails (`eventticketingdb`.`booking_seats`, CONSTRAINT `booking_seats_ibfk_1` FOREIGN KEY (`booking_id`, `event_id`) REFERENCES `bookings` (`booking_id`, `event_id`) ON DELETE RESTRICT)

7. Negative ticket price

```
UPDATE Booking_Seats SET price = -50
WHERE booking_seat_id = 301;
```

ERROR 3819: Check constraint 'nonnegative_price' is violated.

8. Invalid booking status

```
UPDATE Bookings SET booking_status = 'Done'
WHERE booking_id = 201;
```

ERROR 3819: Check constraint 'valid_booking_status' is violated.

9. Seat from a different venue

```
INSERT INTO Booking_Seats VALUES
(999, 201, 9, 101, 500, 'TEST-999');
```

ERROR 1644: Seat belongs to another venue

10. Parent with booking history

```
DELETE FROM Users WHERE user_id = 1;
```

ERROR 1451: Cannot delete or update a parent row: a foreign key constraint fails (`eventticketingdb`.`bookings`, CONSTRAINT `bookings_ibfk_1` FOREIGN KEY (`user_id`) REFERENCES `users` (`user_id`) ON DELETE RESTRICT)

11. Required customer name

```
UPDATE Users SET name = NULL WHERE user_id = 1;
```

ERROR 1048: Column 'name' cannot be null

12. Default values for a new booking

```
INSERT INTO Bookings (booking_id, user_id, event_id)
VALUES (999, 7, 105);
```

Accepted: 1 row(s) affected.

```
SELECT booking_id, total_amount, booking_status
FROM Bookings WHERE booking_id = 999;
```

booking_id	total_amount	booking_status
999	0.00	Pending

13. Valid seat for the same event

```
INSERT INTO Booking_Seats VALUES
(999, 201, 3, 101, 500, 'TEST-999');
```

Accepted: 1 row(s) affected.

```
SELECT booking_id, seat_id, event_id
FROM Booking_Seats WHERE booking_seat_id = 999;
```

booking_id	seat_id	event_id
201	3	101

14. Protect the event venue

```
UPDATE Events SET venue_id = 2 WHERE event_id = 101;
```

ERROR 1644: Venue cannot be changed

Check venue relationships

```
SELECT COUNT(*) AS wrong_venue_rows
FROM Booking_Seats bs
JOIN Seats s ON s.seat_id = bs.seat_id
JOIN Events e ON e.event_id = bs.event_id
WHERE s.venue_id <> e.venue_id;
```

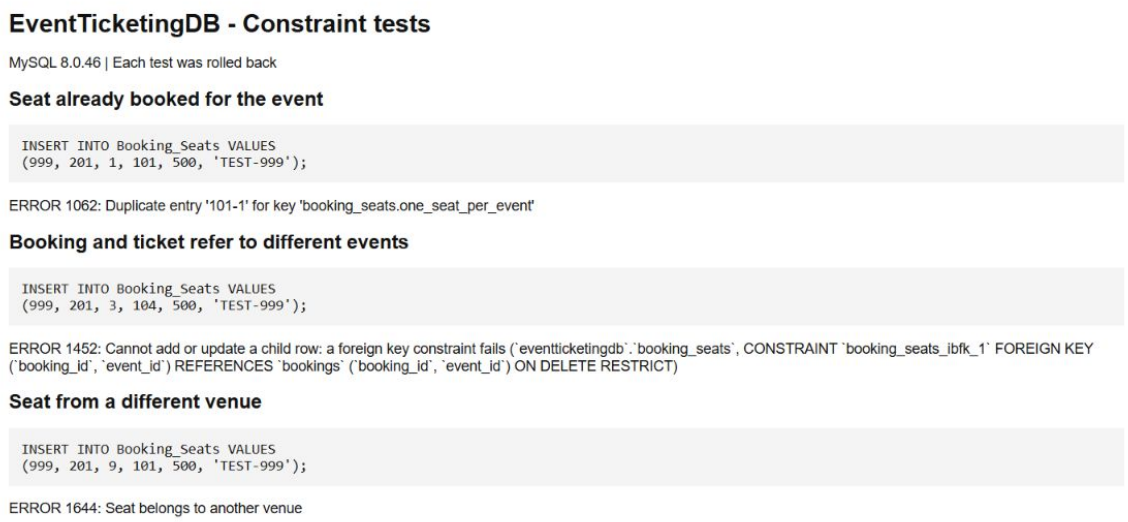
Output

wrong_venue_rows
0

Zero rows have a seat from the wrong venue.

All fourteen tests behaved as expected. The constraints do not automatically update booking totals or process cancellations. Those changes must be made together in a transaction. For a cancelled booking, seat allocations must be removed so the seats can be booked again.

Query result screenshot



Saved MySQL results displayed in the browser.